ECSTORE: Achieving Efficient and Compressible Indexing on Outsourced Encrypted Databases

Tianxiang Shen, Ji Qi, Ning Jia, Haoze Song, Xiapu Luo, Sen Wang, Heming Cui, *Member, IEEE*

***Abstract*—Encrypted Databases (EDBs) are essential for protecting sensitive data outsourced to public clouds, enabling diverse index-based queries over encrypted data. However, existing EDB indexes often incur high storage overhead and performance degradation, primarily due to the poor compressibility of pseudorandom encrypted values, which leads to frequent accesses to slower persistent storage as indexes outgrow main memory.**

We introduce ECSTORE, the first EDB that supports compressible and efficient indexing. Observing that EDB indexes are used solely for lookups and never decrypted, we design ECTREE, a cryptographic hash-based index structure in which each node is a compressible bit-string identifier that conceals plaintext keys. ECTREE enables logarithmic-time encrypted search via a novel membership testing mechanism. To address false positives arising in dynamic workloads, we introduce *Directed View Check* (DVC), which detects inaccuracies and avoids redundant traversals. Additionally, ECTREE's Merkle-tree-like structure supports encrypted query authentication, resisting server compromise. Extensive evaluations show that ECSTORE can achieve up to 94.7% lower latency and 10.5x higher throughput on popular benchmarks compared to notable EDBs.

***Index Terms*—Encrypted database, encrypted search, secure database indexing, index compression.**

I. INTRODUCTION

THE rise of cloud computing has popularized cloud storage systems [1], [2], yet their use of plaintext in processing sensitive user data has raised grave security concerns [3]–[5]. Recent incidents, such as the accusations against TikTok for compromising the data of over 150 million US citizens stored in the cloud [6], and the data breach affecting 2.2 million Pakistani citizens [7], highlight the severe security risks associated with these cloud storage systems.

Recently, *Encrypted Databases* (EDBs) have emerged as a dependable solution for bolstering data security of outsourced cloud storage [8], [9]. For instance, Huawei's GaussDB employs EDBs to safeguard client information in its enterprise ERP systems [10]. Alibaba's ApsaraDB RDS utilizes EDBs to ensure compliance with stringent data protection regulations in the insurance sector [11]. By encrypting data on servers and keeping the decryption keys at the client, EDBs preserve data confidentiality against both privileged database administrators and cloud attackers. To process client queries on encrypted data, notable EDBs (e.g., CryptDB [12] and HElibDB [13]) use diverse cryptographic algorithms [9], [14] to encrypt the primary keys then build indexes on them. State-of-the-art EDB indexes can achieve logarithmic search complexity, equivalent to that of conventional indexes (e.g., B^+-tree).

Despite achieving efficient indexing, existing EDBs often underperform compared to theoretical expectations, especially under dynamic workloads like updates. The main reason is that the EDB size grows almost linearly with database encryption (e.g., at least 5x larger than unencrypted databases [15]). Thus, *compression* [16] becomes essential to fit more data into main memory for efficiency. Moreover, the need for compression intensifies under dynamic workloads as the performance further deteriorates when EDB indexes rapidly expand and saturate main memory (as proved in §VI-A).

Therefore, an ideal EDB should achieve efficient and compressible indexing on dynamic workloads while supporting a full set of operations as conventional unecnrypted indexes.

Much prior work [12], [13], [17], [18] has explored efficient and compressible EDB indexing but they fail to achieve both goals simultaneously due to a fundamental tension between encryption and compression. Specifically, compressing the index before encryption requires the cloud server to have the decryption key to decrypt and decompress the index, exposing plaintext primary keys to compromised servers. Conversely, encrypting the index with keys inaccessible to the server and then compressing it renders classic compression methods ineffective [12], [15], because encrypted data is inherently pseudorandom and lacks redundancy required by compression.

Our key observation to resolve this tension is a fundamental trend in EDBs: *the indexes built on encrypted primary keys are solely used for data lookups and never be decrypted*. This *one-way* nature of EDB indexing, where decryption is forbidden and unnecessary, allows us to use lightweight One-Way Functions (OWFs) [19], such as cryptographic hash functions, for building EDB indexes. OWFs convert keys into one-way *identifiers* that are infeasible to reverse back to plaintext data [19]. Unlike previous EDB indexes built with two-way encryption algorithms that render indexes incompressible due to pseudorandomness, an EDB index built on hash-based identifiers can be organized into a structured form with some redundancy [20], thus allowing for index compression.

Based on this observation, we propose ECSTORE[1], the first EDB to achieve efficient and compressible indexing. At its core is ECTREE, a compressible tree-based index that supports all index operations over encrypted data, including lookups, range searches, inserts, updates, and deletes. ECTREE introduces a new concept of *membership-based* encrypted search by designing a tree structure that stores data membership information using one-way identifiers, and executes searches

T. Shen, N. Jia, and S. Wang are with the Huawei Technologies Co., Ltd. Emails: {shentianxiang6, ning.jia, wangsen31}@huawei.com.

X. Luo is with The Hong Kong Polytechnic University. Email: csxluo@comp.polyu.edu.hk.

H. Song, H. Cui are with The University of Hong Kong. Emails: {hzsong, heming}@cs.hku.hk.

J. Qi is with the Institute of Software Chinese Academy of Sciences. Email: qiji@iscas.ac.cn. *J. Qi is the corresponding author.*

[1]ECSTORE is for an **E**ncrypted and **C**ompressed Data **Store**.

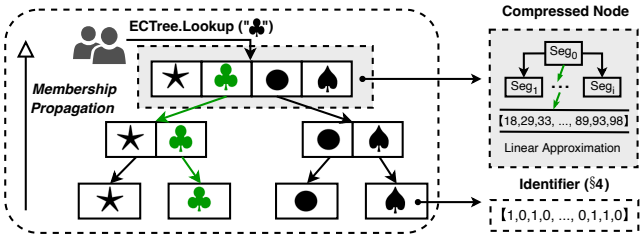


Fig. 1: ECTREE overview. An ECTREE is built by propagating data membership information of identifiers (e.g., ⋆) from the bottom up, and compressing large index inner nodes with linear approximation. It facilitates top-down logarithmic-time search via membership tests.

logarithmically via *membership tests*, without revealing plaintext primary keys.

Figure 1 exemplifies our core indexing idea. An ECTREE is initialized by arranging the identifiers in sorted order at the bottom layer. The membership information of these identifiers is recursively propagated to the root via *union* computations. We term this construction process *membership propagation*, which ensures that each ECTREE node *conservatively* encompasses the membership information of all its children, later facilitating top-down searches by testing the membership of query parameters (e.g., ♣) in identifiers hierarchically.

Membership propagation offers two crucial benefits. First, it enables logarithmic-time search by testing whether query parameters are members (i.e., by determining $\in$ or $\notin$) without revealing plaintext keys. Second, using one-way identifiers allows the structuring of membership information into bit arrays. Since only the non-zero bits are used for encrypted search, we further compress ECTREE by eliminating redundant bits, converting the remaining bits into sorted arrays, and applying linear approximation with space-efficient machine learning models (i.e., learned indexes [21], [22]) over these arrays for efficient compression without sacrificing search functionality.

However, a key performance challenge arises from the issue of *membership hallucination*. Specifically, as more elements are inserted, their membership information accumulates within the ECTREE and propagates toward the root. This accumulation can result in false positives (FPs), where an ECTREE node incorrectly passes a membership test despite not being on the correct search path. This leads to multiple hallucinated search paths and causes redundant traversals. To identify FPs and eliminate redundant traversals, we introduce the *Directed View Check* (DVC) algorithm. DVC ensures that the search target's membership information is exactly propagated in the current ECTREE node by verifying which child nodes have contributed to the propagation, thus confirming the target exists in the node's subtree and preventing unnecessary traversals.

Another challenge is that a compromised server may induce integrity breaches by actively manipulating query results via modification, replay, or deletion [23], [24]. Fortunately, ECTREE's unique construction based on membership information leads to a Merkle-tree-like index structure that can facilitate query authentication by verifying if the requested data is included in the query results. Therefore, through a careful design of ECTREE, ECSTORE provides both data confidentiality and query integrity in a unified manner.

We implemented ECSTORE as a layer above CryptDB [12], a mature EDB that supports encrypted data processing. We developed both ECSTORE-MySQL, a SQL instance based on MySQL [25], and ECSTORE-WiredTiger which is a NoSQL instance based on WiredTiger [26]. Our extensive evaluation shows that ECTREE delivers significant compression while keeping the index functional. We compared ECSTORE with notable EDBs on popular benchmarks, including the TPC-C benchmark [27] for SQL instances and YCSB [28] (with six different workloads) for NoSQL instances. Our evaluation shows that:

- ECSTORE is efficient. ECSTORE achieved 7% to 94.7% lower query latency and 3.7x to 10.5x higher throughput compared to notable EDB baselines (CryptDB [12] and HE-libDB [13]) that support cloud-side encrypted search, by simultaneously achieving logarithmic search complexity and fitting more index components into main memory (§VI-A).
- ECSTORE is secure. For confidentiality, ECSTORE effectively hid plaintexts of keys used for indexing with OWFs, and it resisted attacks that attempted to reverse the keys. For integrity, ECSTORE maintained high effective throughput by detecting all query result breaches, while the baselines failed to detect integrity breaches (§VI-C).
- ECSTORE is scalable. When we increased the number of EDB servers, ECSTORE's throughput increased almost linearly, on both uniform and Zipfian key distributions (§VI-B).

Our main contribution is ECTREE, the first efficient and compressible EDB index. By leveraging membership information, ECTREE supports logarithmic-time encrypted search and enables lossless compression without sacrificing functionality. Additionally, its Merkle-tree-like construction facilitates query authentication. These unique features demonstrate ECSTORE's potential to be deployed in security-critical applications (e.g., anonymous messaging [29] and online streaming [30]). The source code is released on github.com/hku-systems/ecstore.

II. BACKGROUND AND PRELIMINARIES

A. Encrypted Search on Outsourced Encrypted Databases

As organizations increasingly migrate sensitive data to cloud databases, ensuring confidentiality during data processing has become imperative. *Encrypted Databases* (EDBs) [12], [13], [17], [18] have emerged as reliable solutions. Typically, EDBs utilize fully homomorphic encryption (FHE) [31] to encrypt *value* fields, allowing arbitrary computations on an untrusted server without exposing plaintext values. Meanwhile, EDBs employ specialized searchable encryption algorithms [8], [19], [32], [33] to transform *key* fields, encoding primary keys into non-reversible searchable ciphertexts. Then, indexes are built on these ciphertexts to facilitate encrypted search.

Table I summarizes representative EDBs that support encrypted search. mOPE [32] and FH-OPE [33] are two notable EDBs leveraging order-preserving encryption (OPE). OPE ensures that encrypted keys retain the same order as plaintext keys, making it particularly useful for range queries. However, OPE-based solutions face three limitations. First, compressing indexes built on OPE-transformed keys is infeasible because OPE introduces high entropy to encrypted keys [34], which

conflicts with the redundancy requirements of existing compression mechanisms (§II-B). Second, many existing solutions require multiple communication round trips for a single index operation. For instance, mOPE [32] incurs at least $O(\log n)$ interaction rounds per update, leading to significant communication overhead under frequent queries and making query performance highly dependent on network stability. While FH-OPE [33] reduces interaction to a single round per query, it incurs additional security costs to hide search metadata (e.g., query frequency) and suffers from orders-of-magnitude slower performance. Note that, compared to ECSTORE, FH-OPE have an orthogonal goal of achieving metadata security. We further discuss metadata-hiding EDBs like FH-OPE in §VI-D.

The third and most critical limitation of OPE lies in its complexity and implementation challenges. Consequently, existing EDBs, such as CryptDB [12], typically employ OPE exclusively for indexing columns required for range queries, while relying on simpler schemes like deterministic encryption (DET) [9] for other operations (e.g., lookup queries). Specifically, CryptDB generates separate searchable ciphertexts per key using both DET and OPE, maintaining multiple indexes to support different query types. While these indexes offer logarithmic efficiency, CryptDB incurs substantial storage overhead and lacks mechanisms to detect integrity breaches, which is a common threat in public cloud environments [35].

Other notable EDBs adopt different approaches to mitigate OPE's limitations. HElibDB [13] leverages mathematical tools (Fermat's Little Theorem [36]) to evaluate the equivalence of homomorphically encrypted primary keys, eliminating the need for maintaining large incompressible OPE indexes and reducing CryptDB's storage overhead but at the cost of requiring linear scans of the entire database. Dory [8] supports keyword search over encrypted file stores but lacks adaptability for general-purpose databases.

Unlike previous EDBs, ECSTORE aims to develop a compressible EDB index that supports all operations of traditional tree-based indexes, requires only a single round of interaction per query, and remain simple, efficient, and easy to implement.

B. Strawman Encryption-Compression Co-designs

Despite achieving theoretical logarithmic search efficiency, prominent EDBs like CryptDB often underperform because encryption causes nearly linear growth in database size, rapidly saturating main memory (§VI-A). This problem worsens under dynamic workloads with insertions and updates (§VI-B). Although solutions like MiniCrypt [15] propose efficient compression algorithms, they have an orthogonal goal of compressing values rather than indexes on keys. Next, we discuss two strawman approaches that try to co-design encryption and compression in EDB indexes and show their limitations.

One approach is to encrypt a compressed index using techniques like run-length encoding [37] and distributed source-coding [16]. However, both methods require the server to possess the decryption key to decrypt and decompress the index during query requests, risking exposure of plaintext keys if the server is compromised (e.g., by a malicious administrator or cloud-side attacker). Alternatively, the server may delegate the searching task to the client by offloading an entire EDB index to the client, which does not leak sensitive data to the server but incurs substantial communication overhead [8].

System	Plaintext-key Protection	Query Integrity	Index Compression	General Func.	Lookup Complexity
mOPE [32]	✓	✓	✗	✓	*logarithmic*
FH-OPE [33]	✓	✗	✗	✗	*logarithmic*
HElibDB [13]	✓	✓	$n/a.^\star$	✗	*linear*
Dory [8]	✓	✓	✗	✗	*linear*
HybrTC [38]	✓	✓	$n/a.^\star$	✗	*linear*
MiniCrypt [15]	✓	✗	✗	✓	*logarithmic*
CryptDB [12]	✓	✗	✗	✓	*logarithmic*
Z-IDX [39]	✓	✗	✗	✗	*logarithmic*
Blindseer [40]	✓	✗	✗	✓	*logarithmic*
ECSTORE	✓	✓	✓	✓	*logarithmic*

*: $n/a.$ means the EDB does not support indexing.

TABLE I: Characterization of representative EDBs.

An alternative approach is to encrypt the index before applying compression, which is ineffective. This is because encrypted data is highly entropic [34], lacking the redundancy needed for standard compression techniques [20], [41]. To address this, clients might use a dictionary table [42] (i.e., an additional index) that maps uncompressed encrypted keys to compressed codes. However, each data lookup would require the client to find the compressed code locally, while the server linearly scans the EDB to compare codes. This approach is inefficient because it introduces extra reads for compression and decompression.

ECSTORE is designed to harness the advantages of both encryption and compression in EDB indexes while incurring minimal performance overhead. ECSTORE's index design is also independent of workloads and data types.

C. Preliminaries

Index Operations. We aim to devise an EDB index capable of supporting all operations in traditional tree-based indexes. Given x and y as keys and v as value, the index S should support the following operations:

(1) *member(x)* = TRUE if $x \in S$, FALSE otherwise.
(2) *lookup(x)* returns the value associated with key $x \in S$.
(3) *range(x, y)* returns all the elements where $x \leq$ key $\leq y$.
(4) *insert(x, v)* puts an element with key x and value v.
(5) *delete(x)* deletes the element with key x from S.
(6) *update(x, nv)* runs *delete(x)* followed by *insert(x, nv)*.

Bloom Filter. Our new membership-based index is based on the bloom filter [43]. It is a space-efficient data structure for testing whether an element is a member of a set. A bloom filter B is an m-bit array and parameterized by k independently uniform OWFs $\{H_k\}$, each H_i maps an element to a position in B. To add an element x into the filter, we compute its hash using $\{H_k\}$ and set all the bits at the corresponding indices to `"1"`. To test whether an element y is in the set, we again hash y using $\{H_k\}$. If any bits at the corresponding indices in B are `"0"`, y is definitely not in the set (i.e., zero false negatives). Conversely, y is considered to have a high probability of being in the set, albeit with a small configurable false positive rate [43].

It is important to note that while prior EDBs have utilized bloom filters to build indexes, they target specific query scenarios with high complexity. For instance, Dory [8] supports only encrypted keyword searches on file stores with linear complexity. While Z-IDX [39] and Blindseer [40] achieve logarithmic complexity using bloom filters, they overlook the threat of integrity breaches. ECSTORE differentiates itself by aiming to facilitate general queries and provide data confidentiality and query integrity guarantees simultaneously.

Learned Index. Given a key, the learned index [44] maps it to the position in a sorted array of keys, which thus is considered as a trained model. Learned index utilizes a hierarchy of models to locate a key. During a lookup, the higher-level model predicts the model at the next level, and the leaf-level model computes an approximate position of the key in the sorted array. Finally, the learned index employs binary search to rectify inaccurate predictions within an error bound ϵ.

PGM [45] is a space-efficient and updatable learned index. It uses linear models (or *segments*) and divides keys into different segments to approximate the positions of all keys in the sorted array within ϵ distance. PGM's space efficiency stems from the fact that linear models require only constant space (for slopes and intercepts) and constant query time, regardless of the total number of index keys. Each segment in PGM specifies the first key covered by the next-level segment, enabling recursive index construction on the sorted keys of segments at lower levels. To locate a key, PGM predicts positions at each level and corrects them immediately. This process continues until convergence to a leaf-level segment. For insertions, similar to LSM-tree, PGM separates keys into subsets. Each insertion involves finding a series of non-empty sets, merging them into a large subset, and then creating a new index on the large subset. A detailed discussion of PGM can be found in [45].

III. OVERVIEW

A. System Setup

Entities. Same as existing EDBs [15], [25], ECSTORE follows a cloud-hosting model consisting of two roles: the hosting server responsible for providing query services, and the client issuing queries to the server. The client can have multiple endpoint machines sharing the same encryption keys, which are inaccessible to the server.

Data Model. To illustrate our design, we use a simple key-value data model where the EDB stores primary keys and their respective values in two columns. Keys are stored as *identifiers* by hashing the plaintext using OWFs like cryptographic hash function SHA-256 [46], ensuring *key irreversibility* (§III-B). Although cryptographic hashes are deterministic schemes, they are sufficiently secure in our setting as we assume keys are unique and attackers do not collude with clients (§III-B). Values are encrypted using fully homomorphic encryption [47] for arbitrary server-side computations on ciphertexts.

Underlying Database. ECSTORE is designed as a layer over unmodified databases to leverage existing optimizations. We make two assumptions about the underlying database. First, like prior EDBs using bloom filters [39], [40], we assume primary keys are unique. Second, similar to [15], we require the underlying database to provide a conditional atomic update (*update-if*) primitive to support concurrent writes. This primitive is lightweight and has been widely supported in popular databases like Cassandra [48]. It operates as "UPDATE ... IF ..." or "INSERT ... IF NOT EXISTS", executing writes only when a condition is satisfied. Additionally, similar to [8], elements (key-value pairs) are *versioned* to resist replay attacks. Clients may choose to keep some elements synchronized with the server by maintaining their latest versions locally, while others are retrieved from the server as needed. When clients concurrently write to the same element, ECSTORE manages it with update-if primitive and synchronizes versions (§IV-B).

B. Threat Model and Guarantees

Similar to [12], [49]–[51], we adopt a strong threat model in which a malicious adversary $\mathcal{A}$ can compromise servers and arbitrarily deviate from ECSTORE's protocol to disrupt query services. Specifically, $\mathcal{A}$ may passively observe or actively manipulate query results and tamper with stored data through modification, replay, or deletion attacks [24]. We assume that clients are trustworthy and authorized to access all server-side data since they are the data owners. ECSTORE focuses on protecting data against server-side adversaries. Therefore, we assume that $\mathcal{A}$ does not have control over any client and, consequently, cannot execute queries (e.g., insert or delete) through a client to launch pre-image [52] or dictionary attacks [53] aimed at reversing cryptographic hashes.

Out-of-scope Attacks. ECSTORE is not designed to hide side-channel information such as element sizes and data access patterns, which may be inferred through query computations like equality comparisons and sorting operations. While recent cryptographic techniques [54], [55] can mitigate these leaks, they are computationally prohibitive. Thus, addressing side-channel leakage is orthogonal to ECSTORE's goals. However, it is worth noting that unlike prior compression methods (e.g., run-length encoding [37]) that expose data frequency, our compression avoids adding additional side-channel information, thus ECSTORE does not augment such vulnerabilities.

Security Guarantees. ECSTORE ensures data confidentiality through two key guarantees. First, it enforces the irreversibility of the EDB index, meaning that a computationally bounded adversary is infeasible to reconstruct plaintext keys from index node identifiers. Second, it secures data values using fully homomorphic BGV encryption scheme [56] and protects column and table names with AES-256 in CBC mode, which offers semantic security [47]. Additionally, ECSTORE guarantees query integrity by detecting adversarial manipulations, including modifications, replays, and deletions of query results.

C. Architecture

Figure 2 shows ECSTORE's architecture. The server hosts the tree-based EDB index ECTREE in main memory and stores elements at the leaf level physically, similar to conventional indexes like B^+-tree [57]. The server also hosts a list of MAC tags, each corresponding to an element. The client interacts with the server through ECSTORE's query manager module,

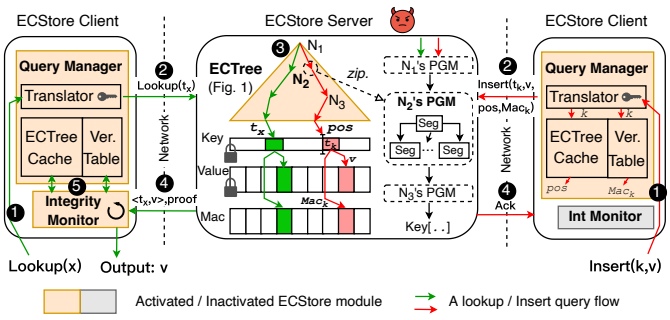


Fig. 2: The architecture and typical workflow of ECSTORE.

consisting of three components. The **translator** converts plaintext query parameters into identifiers for server-side search and maintains cryptographic keys locally. The **ECTREE cache** stores a partial index, including the ECTREE root node for authentication purposes and leaf nodes (primary keys) for write operations (i.e., *insert*, *delete*, and *update*). The local **version table** keeps the latest versions of partial elements for query authentication and write operations. Last, the **integrity monitor** module detects breaches in query results for read operations (i.e., *member*, *lookup*, and *range*) by interacting with the ECTREE cache and version table.

For read operations, like *lookup(x)* in Figure 2, the client converts plaintext x into an irreversible secure primitive called *subfilter* (described in §IV-A), acting as an identifier (denoted as t_x) for encrypted search. The client encrypts the query via AES, signs it, then sends it to the server (❷). The server will drop the connection if malformed queries (e.g., with invalid signatures) are received. Upon receiving a valid query, the server traverses ECTREE with parameter t_x to locate the requested element (❸), returning both the element $< t_x, v >$ and an authentication proof to the client (❹). The client fetches necessary data from local ECTREE cache and version table based on t_x, verifies the proof (❺, detailed in §IV-D), and outputs the decrypted value if verification succeeds.

For write operations, like *insert(k, v)* in Figure 2, the client converts the plaintext key k into the identifier t_k, then finds k's predecessor position pos by looking up the ECTREE cache, and generates the MAC tag Mac_k for the element. The client then sends a query parameterized with t_k, v, pos, and Mac_k to the server (❷). The server updates ECTREE by inserting $< t_x, v >$ at the successor position of pos, adds Mac_k to its Mac list, and finally returns an acknowledgment to the client, indicating the completion of the write operation (❹).

In sum, to the best of our knowledge, ECSTORE is the first EDB that efficiently supports general index operations with logarithmic search complexity and compression capabilities, while providing both data confidentiality and query integrity guarantees (as compared in Table I).

IV. DESIGN

In this section, we first introduce ECTREE's construction, followed by presenting the operations it supports. Then, we discuss how to compress ECTREE. Last, we show how ECSTORE authenticates encrypted queries with ECTREE.

A. ECTREE *Construction*

Observation. We build ECTREE on the key insight that EDB indexes on encrypted primary keys are used solely for data lookups and never decrypted for processing. This allows us to employ lightweight One-Way Functions (OWFs) [19] for constructing EDB indexes. Using OWFs ensures irreversibility of keys, which is sufficiently secure (§III-B).

Opportunity: Bloom Filters. Conventional bloom filters use OWFs like SHA-256 to store hashes of elements as positions in a bit array instead of plaintext keys (§II-C). They efficiently confirm an element's presence within a set, providing unique *data membership* information, but lack indexing capabilities. To leverage this membership information for encrypted search, our preliminary design decomposes a bloom filter into multiple *subfilters*, each storing only the hash of a single element.

Definition 1: Subfilter. *A subfilter is the bloom filter for an individual element, i.e., only one element is assigned to a set for membership testing.*

We use subfilters as the basic units for building ECTREE in encrypted search. As shown in Figure 3, each subfilter serves as an identifier for a plaintext primary key, for example, `"111000"` represents x_2 after applying three OWFs that set bits at positions `0`, `1`, and `2`. These subfilters, all sharing the same array size, are positioned at the bottom layer and arranged in the same order as plaintext keys.

Each inner node in ECTREE is computed by performing bitwise OR on its child nodes' subfilters. Specifically, for an inner node with p child nodes $\{cn_1, \ldots, cn_p\}$ and subfilters of length m, each bit in the inner node's subfilter is computed as:

$$inner[i] \;=\; cn_1[i] \; OR \; ... \; OR \; cn_p[i] \;,\; \forall \; i \in [0, m] \qquad (1)$$

The design rationale behind ECTREE lies in the principle that if any entry in a child node's subfilter contains a `"1"` bit, Equation 1 ensures the corresponding entry in its parent node's subfilter also contains a `"1"` bit. This means a parent node conservatively encompasses the data membership information of all its children because bloom filters verify data membership by testing the presence of all `"1"` bits. We term this approach *membership propagation*, where the membership information of primary keys (as identifiers) is propagated upward to the root. With membership propagation, ECTREE facilitates top-down logarithmic traversal while preserving the irreversibility of converted primary keys inherited from OWFs.

B. ECTREE *Operations*

Lookup Queries. ECTREE determines traversal paths by comparing subfilters, similar to data membership tests in bloom filters. As illustrated in Figure 3, to look up an element (key-value pair), the client converts the plaintext key into a subfilter using OWFs locally. On the server side, ECSTORE traverses ECTREE by verifying at each inner node whether every `"1"` bit in the query subfilter exists in a child node's subfilter. If successful, ECTREE proceeds to that child node because the query parameter is present in that branch due to membership propagation (despite potential false positives, as discussed next). This process continues until reaching the bottom layer,

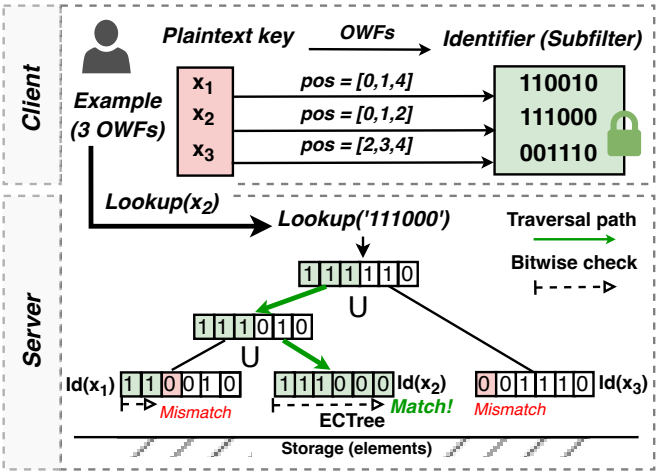


Fig. 3: ECTREE's core concept of membership-based search.

where a full bitwise comparison is performed since each subfilter exclusively represents one key (**Definition 1**). The recursion version of this algorithm is shown in **Algorithm 1**.

Algorithm 1: ECTREE Lookup

```
1  Input: τ: the root node of ECTree, k: the element's key
2  Output: isFound: indicates whether k is found, v: the value
3  Function Lookup(τ, k) do
4      if isLeaf(τ) then
           ▽ Bitwise comp. between subfilters
5          if k.subfilter == τ.subfilter then
6              return <True, τ.value>
7      if isInner(τ) then
8          child[] ← τ's child nodes
           ▽ Test mem. & Detect false positives
9          for each c in child do
10             if τ ∈ c && !IsMemHall(c, k) then
11                 return Lookup(c,k)
12         return <False, Null>
```

Challenge: False Positives Detection. Unlike conventional indexes (e.g., B^+-trees) that use numerical comparisons for traversal, ECTREE relies on data membership information for traversal with subfilter comparisons, which can inadvertently generate false positives (FPs). These FPs can lead to redundant traversals where multiple ECTREE nodes falsely pass membership tests, despite only one correct traversal path existing. We term this phenomenon *membership hallucination*. In the worst-case scenario, this could cause data lookups navigate an exponential number of redundant paths in ECTREE, degrading the complexity from logarithmic to linear.

To illustrate, Figure 4 shows an example of membership hallucination. Assuming the utilization of 6-bit subfilters with three OWFs, for clarity, we use subscripts to denote the positions of `"1"` bits within each subfilter (e.g., $[245]$ represents a subfilter `"001011"`). Suppose a client submits a lookup query parameterized with $[024]$. Ideally, an ECTREE should follow the path: $N_1 \rightarrow N_3 \rightarrow N_6$ (in green). However, N_2 falsely passes the membership test for $[024]$ because the 0th, 2nd, and 4th bits of N_2 are confirmed as `"1"`, so ECTREE mistakenly searches the entire left branch rooted at N_2 (in red), leading to redundant comparisons of N_4 and N_5.

We observe that false positives stem from *undirected* membership propagation. Specifically, in Figure 4, N_2 inadvertently includes $[024]$ by uniting its child nodes's subfilters, resulting in an FP where `"1"` bits originate from multiple child nodes. To detect such false positives, we augment each ECTREE node with a *direction view* indicating the source of every `"1"` bit. For example, the view of N_2 shows the 4th bit originates from its left child, the 2nd bit from both child nodes, and the 0th bit from its right child. By analyzing this direction view, ECTREE will stop traversing N_2's branch even if N_2 contains the requested $[024]$ because the 0th bit and 4th bit originate from different sources, indicating that the query parameter is not present in N_2's subtree.

Based on this observation, we propose the *Directed View Check* (DVC) algorithm to identify false positives and eliminate redundant index traversals for efficient encrypted searches in ECTREE. Specifically, the direction view of an ECTREE node is formed as a table T, where rows represent subscripts of `"1"` bits in node N's subfilter and columns denote identifiers of child nodes containing these specific `"1"` bits. DVC takes the query parameter's subfilter Sub as input and determines whether Sub is a hallucination of N by verifying the *Common Child Set* (CCS) for Sub in T.

Definition 2: ECTREE's Common Child Set (CCS). *Given a requested subfilter Sub, the CCS of an* ECTREE *node N comprises a collection of N's child nodes, wherein each node in the set encompasses all `"1"` bits present in Sub.*

To compute the CCS for node N with parameter Sub, DVC enumerates the child nodes and checks corresponding bits. Specifically, if all the `"1"` bits in Sub are present in a child node's subfilter, DVC adds that child's identifier to the CCS. To determine the occurrence of a hallucination, DVC checks the size of the resulting CCS: an empty CCS means none of N's child nodes encompass all the `"1"` bits in Sub, indicating that N is not on the traversal path; otherwise, N is on the traversal path. **Algorithm 2** illustrates this methodology.

In summary, ECTREE supports encrypted lookup queries by comparing subfilters, employing DVC to detect FPs (hallucinations) of comparisons, and pruning redundant traversals (**Algorithm 1**, line 10). Consequently, ECTREE, as a tree-based index, achieves logarithmic-time search complexity relative to the size of the EDB.

Index Inserts. In addition to lookup queries, ECTREE supports insert operations while preserving strict index order by maintaining ordered subfilters at the leaf level for range operations. Since subfilters are inherently unordered identifiers, we adopt a client-assisted design where the client maintains a small cache (much smaller than ECSTORE) that stores essential information for encrypted insertions. Specifically, this cache retains a partial index comprising root nodes and the plaintext keys associated with leaf-level subfilters. Multiple root nodes may exist in the cache because the server may manage several ECTREEs due to index adjustments (discussed later). A cached root node facilitates element insertion into a specific ECTREE and enables query authentication for detecting query integrity breaches (§IV-D).

To insert an element with key k, the client retrieves a root node r and the predecessor of k from its local cache, then transforms the predecessor into a subfilter S_{pre}. The server-side ECTREE (rooted at r) locates S_{pre} at the leaf

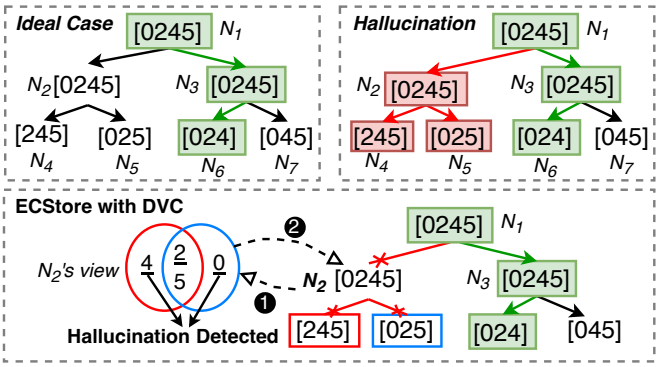


Fig. 4: Searching an ECTREE may have false postives due to membership hallucinations. We propose the DVC algorithm to promptly detect hallucinations and eliminate redundant traversals.

Algorithm 2: DVC (ECTREE Hallucination Detection)

```
1  Input: N: node of ECTree for check, k: element's key
2  Output: isFound: indicates if k is a hallucination of N
3  Function IsMemHall(N, k) do
4      child[] ← N's child nodes
5      pos[] ← positions of "1" bits in k's subfilter
6      isMem ← True
7      for each c in child do
8          for each p in pos do
9              if c.subfilter[p]! = "1" then
10                 isMem ← False
11                 break
           ▽ No hallucination is detected in node N
12         if isMem then
13             return False                ▷ Algorithm 1 continues
14         isMem ← True
15     return True
```

layer and inserts the element next to S_{pre} to keep the leaf nodes ordered. Upon successful insertion, the server returns an acknowledgment to the client.

Analysis of ECTREE Cache. We analyze the space complexity and practical storage requirements of the ECTREE cache, showing that the client incurs a reasonable storage cost while benefiting from ECSTORE's encrypted search service.

Theoretically, the ECTREE cache requires linear space, placing it asymptotically in the same complexity class as existing secure tree structures [33], [58]. Specifically, in our design, if there are N subfilters, the ECTREE cache stores $O(N)$ keys. Even when accounting for root nodes, the total storage remains a constant factor less than N, which is still $O(N)$ asymptotically. For comparison, in non-recursive Path ORAM [58], a famous tree structure for secure data access, the client maintains a position map that assigns each of the N data blocks to a leaf label in the ORAM tree, requiring $O(N)$ map entries, along with a small stash of $O(logN)$ blocks. Since the client's storage in Path ORAM is dominated by the position map (which is $O(N)$), the additional $O(logN)$ stash represents only a minor additive term. Similarly, FH-OPE [33], an OPE-based EDB, also necessitates $O(N)$ client storage to facilitate server-side index operations on encrypted data. Overall, the theoretical client storage cost of the ECTREE cache aligns with that of other well-known secure tree structures.

In addition to having the same asymptotic complexity as existing secure tree structures (e.g., non-recursive Path ORAM), we implement runtime optimizations to further reduce client-side storage costs. Specifically, since the majority of the ECTREE cache consists of plaintext keys that can be compressed, we leverage CompressDB [59]'s context-free data compression to significantly shrink the cache size, from 762 MB for 100 million elements with 8-byte keys to 143 MB. This reduction brings the storage requirement to a level typically considered acceptable for client usage in public cloud services [60].

One might consider replacing ECTREE with a combination of a hash map to support range queries. In such a design, the hash map is used to locate the positions of boundary keys (e.g., range start and end) in $O(1)$ time, followed by a sequential scan over the sorted array to retrieve matching records. While this hybrid approach can be effective under ideal conditions—such as uniform key distributions—it has several limitations. In practice, hash collisions may occur, and some implementations (e.g., Java's HashMap) convert collision-heavy buckets into balanced binary trees, resulting in $O(log(N))$ lookup time. ECTREE offers the same time complexity as the balanced tree index. As a Bloom filter-based hierarchical index, ECTREE enables logarithmic-time search and supports early termination for negative lookups: if a queried key is absent from the Bloom filter at an internal node, the traversal can be safely aborted without descending to the leaf level. This significantly reduces unnecessary computation. Moreover, ECTREE provides Merkle-style authentication of query results, ensuring verifiable correctness even in adversarial or untrusted storage settings.

Concurrent Encrypted Writes. To prevent concurrent clients from overwriting each other's updates, we rely on the widely available update-if primitive in modern databases, as inspired by Zheng et al. [15]. The core idea is that an ECTREE root summarizes the membership information of all keys, allowing the client to create a snapshot of the current database using the root at the time of write submission. The server then verifies this root using update-if to ensure it writes to the correct data.

Specifically, when client $C1$ submits a query to update element e, he retrieves the root r from its cache and issues an update-if query parameterized with $<e, r>$ to the server, specifying that e should be updated only if the ECTREE root remains r. If the server detects its current root differs from r, indicating that another client has modified the database, $C1$ will synchronize its cache with the server then resubmit a new write query parameterized with the latest root. After finishing the write, ECTREE backtracks along the traversal path in the opposite direction and updates the subfilters (using bitwise-OR) and direction views associated with the nodes along this path. Next, ECTREE will judge whether it should trigger an adjustment to keep its FP rate bounded, as discussed next.

Note that, ECSTORE is mainly designed for read-intensive scenarios and most suitable for applications that prioritize read performance and require end-to-end data confidentiality, such as financial statistics [61] and medical analysis [62]. Exploring more efficient concurrent encrypted writes in ECSTORE could be a promising future direction.

ECTREE Adjustment. While DVC (Algorithm 2) effectively detects FPs in static, read-only settings, its effectiveness diminishes under dynamic workloads (e.g., insertions and updates). Excessive insertions cause membership information ("1" bits)

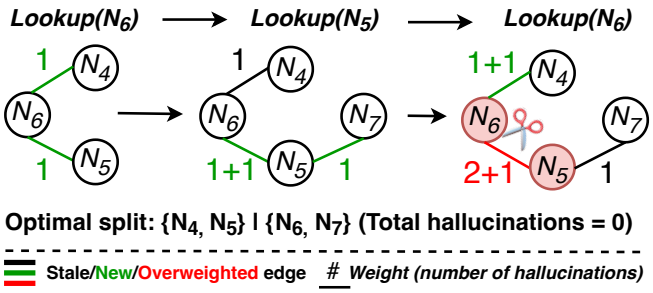


Fig. 5: An ECTREE adjustment example. ECTREE undergoes a split when an excessive number of hallucinations are detected by DVC.

to accumulate at higher-level nodes, causing frequent FPs and increasing the DVC overhead. To maintain a bounded FP rate under dynamic workloads, ECTREE should be split when certain conditions are met.

• ***When to adjust?*** We propose two main criteria to decide whether to trigger adjustment on an ECTREE.

First, the number of inserted elements in an ECTREE rooted at r is at least α times larger than elements in the last adjustment, expressed as $\frac{r.insert_num}{r.total_num} \geq \alpha$, where $r.insert_num$ increments with each insertion and $r.total_num$ is the number of elements by the time of the last adjustment. α is set to 2 by default, derived from the logarithmic methods of PGM [45], i.e., always triggering the split process for an index when the inserted elements have doubled. This criterion effectively reduces the frequency of adjustments.

Second, we trigger an adjustment when the ratio of DVC-detected hallucinations on node n to the number of insertions in the ECTREE rooted at r exceeds a threshold β, formulated as $\frac{n.hall_num}{r.insert_num - r.total_num} \geq \beta$. Excessive hallucinations degrade performance by requiring additional view checks. By default, we set the threshold β = 0.1 to achieve an appropriate adjustment rate. To further choose proper parameters, we study the parameter sensitivity for α and β in Figure 14.

• ***How to adjust?*** Upon detection of hallucinations, the nodes related to hallucinations are cataloged into an *edge-weighted graph*. For the major (second) condition, ECTREE adjustment is triggered by an overweighted edge, which indicates that an excessive number of hallucinations are accumulated.

Figure 5 exemplifies the procedure of an adjustment. Nodes in Figure 5 ($N_4 \sim N_7$) are from the ECTREE in Figure 4. Consider three lookups are conducted after a series of insertions, ECSTORE detects and records hallucinations for each lookup using DVC. For instance, *Lookup(N_6)* (i.e., *Lookup([024])*) will create two edges $\{N_6\text{-}N_4\}$ and $\{N_6\text{-}N_5\}$ weighted 1, because N_4 and N_5 propagate a hallucination of N_6 when searching [024]. Similarly, the next lookup (i.e., *Lookup(N_5)*) creates a new edge $\{N_5\text{-}N_7\}$ and increments the weight of $\{N_6\text{-}N_5\}$. Finally, when *Lookup(N_6)* further increments the weight of $\{N_6\text{-}N_5\}$, this overweighted edge triggers a split.

Next, we determine the optimal split method by applying Kruskal's algorithm [63] to compute the Minimum Spanning Trees of the graph, aiming to minimize the total number of hallucinations (weights) in each split ECTREE. In Figure 5, ECTREE is divided into two groups of nodes, $\{N_4, N_5\}$ and $\{N_6, N_7\}$, and two new ECTREEs are generated separately. This split minimizes the total weights (hallucinations) in the resulting trees; in this case, the total weights are zero since there are no edges between $\{N_4, N_5\}$ and $\{N_6, N_7\}$.

More Operations. *Range queries* are supported by converting the range parameters into subfilters, locating the corresponding leaf nodes using lookup operations, and retrieving all elements positioned between these leaf nodes like a conventional B^+-tree. *Delete queries* are supported by looking up the entry of the element and temporarily marking that entry as NULL; subsequent lookup queries for this deleted key result in traversal to a NULL entry, indicating the absence of the key. We integrate a lazy update strategy to enhance the query performance by periodically clearing NULL entries for batches of deletions (see our implementation optimization in §V-A).

Join queries are supported due to the deterministic encoding methods used by ECSTORE. Specifically, all keys are stored as subfilters for indexing by deterministically hashing plaintext keys; column and table names are deterministically encrypted using AES-256 CBC mode. Such determinism enables general join queries. For instance, consider the following join query:

```
SELECT Customers.CustomerID FROM Customers
INNER JOIN Orders USING CustomerID
```

In this example, the `USING` clause automatically applies the condition Customers.CustomerID = Orders.CustomerID. Since the table and column names are deterministically encrypted as AES ciphertexts, and the elements in the columns are deterministic subfilters, ECSTORE processes the join by searching *AES(Customers)*, *AES(Orders)*, and *AES(CustomerID)* to locate the target tables and columns. It then performs equality comparisons between *Subfilter($CustomerIDs$)* in two tables. This approach generalizes to other types of join queries, including left, right, and full joins. Handling cross join queries is straightforward, as ECSTORE computes the Cartesian product of the two tables by combining every row from the first table with every row from the second table, directly on the AES ciphertexts, without the need for subfilter comparisons.

C. ECTREE *Compression*

As analyzed in §II-B and evaluated in §VI-A, encryption brings significant overhead to index storage. However, prior research (e.g., OPE-based and FHE-based EDBs) has failed to balance efficient encrypted search with index compression. The key challenge is that traditional two-way encryption generates highly entropic pseudorandom data [34], which lacks the redundancy necessary for effective compression [20].

Unlike previous cryptographic indexes, we exploit the one-way property of EDB indexing, where decrypting primary keys is forbidden (§IV-A), to construct subfilters. A subfilter is an m-bit array, parameterized by k independently uniform OWFs $\{H_k\}$ like SHA-256; each H_i maps an element to a position in the array (totally k positions) as either `"1"` or `"0"`. A subfilter inherits all properties of bloom filter except that it holds only one element (**Definition 1**). Initially, subfilters can be compressed using offset compression (***Step 1***). However,

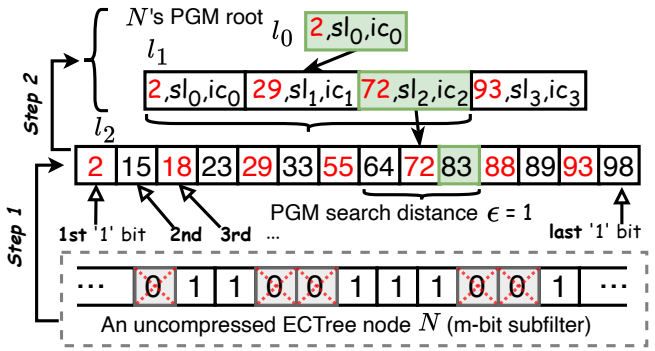


Fig. 6: A lookup example in a compressed ECTREE node.

offset compression alone is insufficient, as the ECTREE size still grows with the total number of indexed keys, which is undesirable in a dynamic EDB with insertions and updates.

To further enhance ECTREE compression (***Step 2***), we draw inspiration from learned indexes like the PGM index [45]. PGM is a space-efficient range index that approximates key distribution in a sorted array to determine key positions. Its space efficiency stems from storing only the slopes and intercepts of several linear models, requiring constant space regardless of the total number of keys, which is crucial in our setting. Next, we demonstrate ECTREE compression using these two steps and give a running example of data lookup on the compressed but fully functional ECTREE.

• ***Step 1: Selective Bit Extraction.*** Recall that ECTREE nodes are hash-based, structured bit arrays (subfilters), same as conventional bloom filters, only the `"1"` bits in subfilters are checked for membership tests (§IV-B). Thus, we convert each subfilter into a *sorted* offset array by extracting the positions of only these `"1"` bits. This conversion discards unnecessary `"0"` bits and significantly saves storage space.

• ***Step 2: Hierarchical PGM Lookup.*** Despite reducing redundant bits and forming sorted offset arrays in **step 1**, ECTREE's size still grows exponentially with the number of inserted elements due to accumulating offsets from membership propagation. To address this, we leverage the PGM index to compress each sorted array (and thus the entire ECTREE) into constant space occupancy.

As depicted in Figure 6, an ECTREE node is compressed into a PGM index, which contains a hierarchy of linear models (segments). These segments divide the offsets of the node into different segments to approximate the positions of all offsets in the sorted array, within ϵ distance. Each segment in the PGM delineates the first key (in red) covered by subsequent segment at the next level. To locate a specific key, PGM predicts positions at each level and instantly rectifies any discrepancies through a binary search within $\pm\epsilon$ distance. This process continues until convergence to a leaf segment.

Figure 6 shows the lookup procedure on a compressed ECTREE node. Instead of comparing every `"1"` bit over uncompressed subfilter, ECTREE searches the offsets of these bits over the space-efficient PGM. Suppose the PGM has a search distance $\epsilon = 1$ and the offset being searched is $k = 83$. The search begins from the root segment, i.e., a linear model with a slope sl_0 and an intercept ic_0, the next position

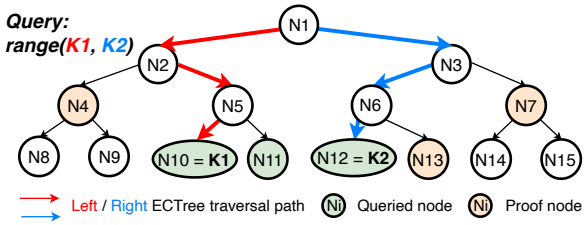


Fig. 7: A range query authentication example with ECTREE.

$\lfloor k * sl_0 + ic_0 \rfloor$ is computed to be 1 for the next level. The search then proceeds to locate k within the range $[1-\epsilon, 1+\epsilon]$ in l_1, including the offsets [2, 29, 72]. It is determined that the next segment for the search is $l_1[2]$ because $k > 72$. Then, the position for the next (leaf) level, l_2, which represents the extracted offset array, is computed as 8. Finally, a binary search is performed to search for k within the range $[8-\epsilon, 8+\epsilon]$ in l_2. Finally, it is found that k is located at position 9 in l_2.

In a compressed ECTREE, update operations require cascading updates for a sequence of PGM indexes of the nodes along the traversal path. Each PGM executes updates by identifying a series of non-empty sets of offsets, merging them into a subset, and then initializing a new segment based on this expanded subset. Our evaluation shows that the overhead of PGM updates in compressed ECTREE is moderate for write operations, primarily because the PGM uses only simple linear models, which is fast for model updating (§VI-A).

D. ECTREE*-driven Query Authentication*

So far, we have assumed that the server adheres to our protocol in an honest-but-curious manner. Next, we show how we leverage the unique ECTREE structure to detect malicious manipulations, including modifications, replays, and deletions of query results by enforcing two authentication rules.

• ***Rule 1: Freshness Validation.*** This rule serves to detect modifications and replay attempts using MACs. The construction of MAC must satisfy two conditions. First, it should only be valid for the latest update to prevent replay attacks. Second, the MAC should be specific to the respective element and not apply to other elements. To achieve this, we compute the MAC over both the subfilter and the version of the element obtained from the client's local version table. With each write query, a MAC tag is piggybacked to the query and stored in the server. After a client retrieves query results, he verifies the MAC tag to ensure the results are not modified or replayed.

• ***Rule 2: Completeness Validation.*** This rule aims to detect deletions by recomputing the ECTREE root through retrieved proofs and comparing it with the locally stored original root, like a Merkle tree [64]. Using a range query as an example, upon completing the data lookup, the server generates a proof by fetching the subfilters of all left sibling nodes along the left traversal path and the subfilters of all right sibling nodes along the right traversal path. The server piggybacks the proof to the result and transmits it to the client.

Upon receipt of the proof, the client recomputes the ECTREE root in a Merkle-tree fashion using the requested query results and proofs (sibling nodes in traversal paths), then compares the recomputed root with the original root stored

in the local cache, which was previously stored by the client and synchronized for each element write and index adjustment. Figure 7 exemplifies the procedure: given a range query that searches for elements in $[K_1, K_2]$, the server returns the queried nodes $\{N_{10}, N_{11}, N_{12}\}$ along with several proof nodes $\{N_4, N_7, N_{13}\}$ for recomputing the root N_1. The client verifies the completeness of results by recomputing N_1 using the returned data. Any mismatch indicates that the results are incomplete, similar to the principle of Merkle tree verification.

V. Implementation and Security Analysis

A. Implementation Details

We implemented ECSTORE with 5021 lines of C++ code on CryptDB [12], a modular framework for evaluating EDBs. We utilized asynchronous RPC calls [65] to send ECSTORE's protocol messages, and these messages were encrypted via AES [66]. We modified CryptDB's client to spawn two new threads: one thread calls `QManager_sync()` to update the ECTREE cache and version table for dynamic workloads (e.g., inserts and updates). We adopted the widely used *update-if* primitive to let the server efficiently handle asynchronous updates from multiple clients (§IV-B), ensuring eventual consistency—a practice commonly observed in cloud-hosting databases due to its performance benefits [67]. The second thread executed `Int_verify()` to conduct result verification of queries, actively engaging ECSTORE's integrity monitor module, and fetching proof (i.e., ECTREE roots and versions) from the query manager module as required.

Case Study. We built two EDBs on MySQL [25] and MongoDB's default NoSQL key-value backend, WiredTiger [26], by replacing their default B^+-tree and B-tree indexes with ECTREE. This replacement is enabled by ECSTORE's unclustered design, which separates index design from underlying storage, making it adaptable to different data stores and able to leverage their optimizations (§III-A). Notably, WiredTiger maintains an LRU cache for its entire B-tree traversal path, including both inner and leaf nodes' pages. To comply with such caching mechanism, ECSTORE persisted the traversed ECTREE pages, allowing WiredTiger to cache them for optimized performance.

Runtime Optimization. First, to enhance query performance for deletions, we implemented a lazy update strategy. Instead of immediately updating after each deletion, a resource-intensive process involving root subfilter recomputations, ECTREE marked deleted elements as NULL entries and periodically updated itself in batches. Correctness is maintained by storing a supplementary bloom filter for deleted elements called `DBF`; each query performs a `DBF.check()` to confirm whether the elements have been deleted. Second, ECSTORE included a tool to empirically determine the optimal LRU cache size for caching index node pages in WiredTiger. By generating a throughput-versus-cache-size graph using representative datasets and workloads, similar to [15], ECSTORE selected the cache size that maximizes throughput. Third, to reduce the storage space required by the client-side cache (originally about 762 MB for 100 million elements with 8-byte keys), we integrated CompressDB [59]'s context-free data compression. By doing so, we reduced the cache size to 143 MB, which is normally considered acceptable for client usage in public cloud services [60].

B. Security Analysis

The following theorem captures ECSTORE's security:

Theorem 1. *Given the mathematically irreversible nature of the one-way functions (specifically, cryptographic hash functions like SHA-256), the uniqueness of primary keys, the non-collusion between the server and client, and the semantic security of the homomorphic encryption algorithm and MAC algorithm, the* ECSTORE *protocol ensures the mathematical irreversibility of primary keys, semantic security for values, and resilience against integrity breaches in the presence of a malicious adversary $\mathfrak{A}$ on the cloud-side.*

The above theorem is established through the following two key lemmas: **Lemma 1** shows the impracticality for an adversary $\mathfrak{A}$ to reverse-engineer ECSTORE's index keys (i.e., subfilters) used for encrypted search; **Lemma 2** confirms the robustness of ECSTORE in detecting any integrity breaches conducted by $\mathfrak{A}$.

Lemma 1 (Irreversibility of ECTREE Keys)**.** Given an ECTREE node, i.e., a subfilter sf, and a set of cryptographic hash functions H, it is computationally infeasible to find any input x such that $H(x) = sf$.

Proof Sketch. We establish this lemma by considering two critical aspects. First, the server's inability to impersonate the client prevents it from launching dictionary attacks by attempting to generate hashes for arbitrary inputs to match stored hash values. Second, due to the assumption of uniqueness of keys and the server's inability to manipulate primary keys, pre-image attacks become infeasible; without access to the original keys, the server cannot derive their hashes to bypass the irreversibility property of the cryptographic hash function. The combination of a pre-image-resistant cryptographic hash function, unique primary keys, and the server's inability to collude ensures the irreversibility of hashed primary keys. Thus, cloud-side adversaries cannot derive plaintext keys by observing stored subfilters, i.e., it is computationally infeasible to find any input x such that $H(x) = sf$.

Lemma 2 (Integrity of ECSTORE's Query Results)**.** ECSTORE is robust to detect any server-side malicious attempts to manipulate query results, including modifications, replays and dropping (deletions).

Proof Sketch. To prove this lemma, we validate the completeness of ECSTORE's two authentication rules for detecting any integrity breaches. First, according to ECSTORE's completeness rule (**Rule 2**), if S represents the set of search results, and l and r denote the query range boundaries, the rule verifies that $max\{x \in S : x < l\} \notin S$ and $min\{x \in S : x > r\} \notin S$, guaranteeing that no elements within the queried range are omitted. Additionally, any modifications or omissions in the results trigger a failure to reconstruct the root subfilter (§IV-D). Second, let $\tau = \mathrm{MAC}(S, v)$ be the MAC of S with the version number v held by the non-colluding client. The freshness rule (**Rule 1**) ensures that any response with a different τ can be identified. This rule is inherently secure due to the semantic security of the MAC scheme [68]. By combining

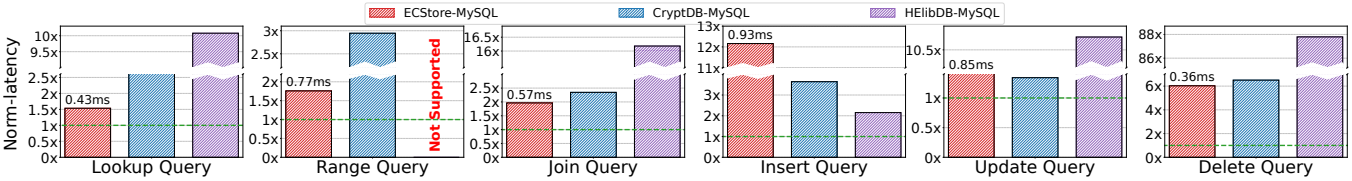


Fig. 8: Normalized end-to-end latency of running TPC-C on all baselines in SQL instances. All systems' latency results are normalized to vanilla unencrypted MySQL (in green dashed line). Values on each red bar indicate ECSTORE's average latency.

TABLE II: YCSB workload description. **L**, **U**, **I**, and **R** denote lookup, update, insert, and range query, respectively.

YCSB	A	B	C	D	E	F
Type	L : U	L : U	L	L : I	R : I	L: I
Ratio (%)	50:50	90:10	100	95:5	95:5	10:90

the two rules, ECSTORE effectively detects any modifications, deletions, dropping, or replay attempts conducted by adversary $\mathfrak{A}$ through validation of the returned proof from the server with the assistance of ECTREE, completing the proof.

To sum, **Lemma 1** proves that $\mathfrak{A}$ is infeasible to reverse the ECSTORE's index keys and **Lemma 2** proves that any integrity breaches conducted by $\mathfrak{A}$ can be detected in ECSTORE. Together these complete the proof of **Theorem 1**.

VI. EVALUATION

Testbed. Without explicit mention, we used one server machine and (up to) 10 client machines. Each machine has a 2.60GHz Intel E5-2690 V3 CPU, 64GB memory, 40Gbps NIC, and 24 cores. The average ping latency between the nodes was set at 0.17ms using Linux traffic control [69].

Baselines. We implemented ECSTORE on CryptDB [12], a mature framework for evaluating EDBs. We built two EDBs with ECSTORE by augmenting prominent databases: ECSTORE-MySQL and ECSTORE-WiredTiger. MySQL [25] is the most prevalent SQL backend for EDBs; WiredTiger [26] is a popular key-value store that is the default NoSQL backend for MongoDB. We compared ECSTORE to vanilla unencrypted databases (i.e., MySQL and WiredTiger without encryption) and three encrypted baselines (with no compression support): CryptDB [12], HElibDB [13], and uncompressed ECSTORE (UC-ECSTORE). CryptDB builds multiple cryptographic indexes and supports all index operations without compression. HElibDB employs Fermat's Little Theorem [70] to directly compare database elements without indexes but requires scanning the entire database.

For an apple-to-apple comparison, we enhanced the baselines as follows. First, all baselines were extensively implemented on both SQL and NoSQL instances. Second, we integrated optimizations from vanilla unencrypted databases (e.g., index-compatible LRU caching) into all baselines to stimulate their performance. Due to space limitations, a detailed discussion of implementation and runtime optimization is placed in §V-A.

Workloads. We evaluated both SQL and NoSQL workloads employed by notable EDBs, which include the TPC-C [27] for SQL instances and YCSB [28] for NoSQL instances.

Table II provides a summary of the YCSB workloads (A-F) that we evaluated. We mainly focused on YCSB as it contains diverse types of workloads [28], including update heavy (A), read mostly (B), read only (C), read latest (D), range scan (E) and write mostly (F). In line with [71], for the NoSQL instances, we initiated with 100 million elements (key-value pairs), necessitating an 8-level tree-based index in all baselines, excluding HElibDB; each element had an 8-byte key and 64-byte value since small requests are prevalent in real-world workloads [72]. In this default workload, ECSTORE requires only 143MB of cache space on the client side, a significant optimization from the initial 762MB achieved through the utilization of context-free compression [59] (more details in §V-A). We evaluated both Uniform and Zipfian key distributions on YCSB-C for scalability tests. For YCSB-D, YCSB-E, and YCSB-F, the inserted keys were uniformly and randomly generated, with the client configured to query recently inserted keys. Each experiment was run for 60s and we collected results in 15s∼45s to avoid the disturbance caused by system start-up and cool-down. Logging was disabled in all systems. We focus on these questions:

§VI-A How efficient is ECSTORE compared to baselines?
§VI-B How scalable is ECSTORE?
§VI-C How robust is ECSTORE under integrity breaches?
§VI-D What are the lessons we learned?

A. End-to-end Performance

We first compared the performance of ECSTORE and baselines in a fault-free scenario where no integrity breaches occurred in the query results. Figure 8 shows that ECSTORE achieved 84.2%∼94.7% lower latency compared to HElibDB. This is attributed to the use of ECTREE index, which provides logarithmic search complexity as opposed to HElibDB which requires linear scans to search for equivalent encrypted data.

Priority: Read Query Latency. Figure 8 further shows that HElibDB exhibited 4.5x and 4.8x lower insert latency than CryptDB and ECSTORE (although ECTREE's insert-induced adjustment is rare and the time used in adjustments is relatively small, as shown in Table IV). This is because HElibDB simply appends inserted elements to its storage, and it does not require building indexes on encrypted data, thereby accommodating more elements in main memory and reducing the frequency of writes to persistent storage. In contrast, both ECSTORE and CryptDB prioritize read performance by maintaining sorted indexes for logarithmic encrypted search.

For update queries (with 90% value updates and 10% key updates in TPC-C), ECSTORE recorded 1.2x higher latency than CryptDB. This is mainly because ECSTORE uses ML models (i.e., PGM [45]) for compression; each update operation necessitates updating a sequence of models along the traversal path (§IV-C). This shows that using simple ML

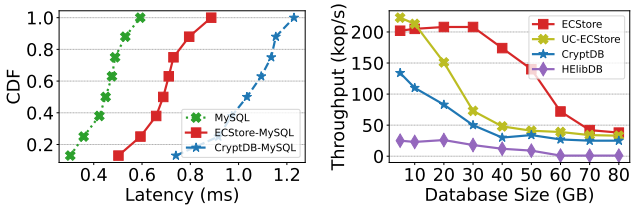


Fig. 9: (a) CDF of range query latency on TPC-C. (b) Lookup query throughput in NoSQL instance (WiredTiger) on YCSB.

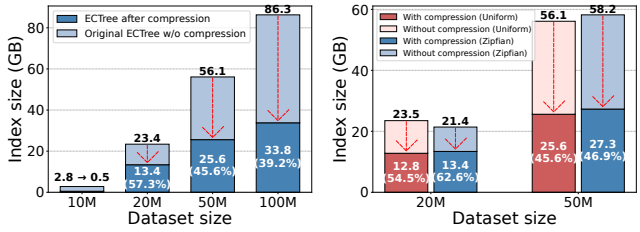


Fig. 10: (a) Compression ratio of ECTREE on different dataset sizes. (b) ECSTORE consistently achieved a high compression ratio under different types of workloads.

models for compression incurs moderate overhead but delivers substantial gains in read performance, as shown next.

Breakdown. Compared to CryptDB, ECSTORE achieved up to 50.6% lower read latency on the TPC-C workload (from 7.7%) and up to 1.9x higher read throughput on the YCSB workload (from 1.3x), as shown in Figure 11. This result highlights that although both ECSTORE and CryptDB support a full set of index operations with the same logarithmic complexity, ECSTORE achieves notable performance improvements through its compression, which reaches up to a 57.3% ratio (as shown in Figure 10).

Table III shows the breakdown of lookup and range queries running on the TPC-C. CryptDB's primary overhead was observed to be both cryptographic operations and encrypted search over uncompressed indexes. Despite ECSTORE incurring an additional overhead of 46.2% to 54.5% for query authentication, it still surpassed CryptDB in performance. We further discovered that compared to CryptDB, ECSTORE achieved only 18.1% and 15.6% lower lookup and range query latency respectively, without utilizing the DVC algorithm (detailed in §IV-B), even ECSTORE leverages index compression. This reveals that the incorporation of DVC for detecting false positives in static read-only workloads effectively mitigates redundant traversals and improves both latency and search throughput, as proved in Figure 11 and Figure 12. In high-update scenarios, the overhead of DVC may increase due to the growing size of the direction view maintained in each ECTREE node (see Figure 13). However, this overhead remains low, as ECSTORE employs a hashmap-based implementation for fast view checks with $O(1)$ complexity. When datasets become denser (Figure 15(b)), ECSTORE's performance slightly decreased. The increasing of elements leads to higher occupancy in subfilters and increases the chance of false positives, causing more traversal. Meanwhile, parallelizing the DVC algorithm further accelerates the process, with the false positive detection rate remains unaffected.

Table III also shows that ECSTORE exhibited a more significant performance improvement in range queries, as depicted

TPC-C Query Latency (in milliseconds)

Systems	ECSTORE		CryptDB	
Query type	LQ	RQ	LQ	RQ
P1 (*client*): Parameter transform	0.12	0.16	0.39	0.52
P2 (*server*): Data lookup	4e-4	1e-3	0.19	0.39
P3 (*client*): Query authenticate	0.2	0.42	*N/S*	*N/S*
P4 (*client*): Result decrypt	0.11	0.19	0.18	0.65
End-to-end ($\sum P_i$)	0.43	0.77	0.76	1.56

TABLE III: Comparison of query latency on the TPC-C workload. **LQ** and **RQ** mean lookup and range query. *N/S* means not supported.

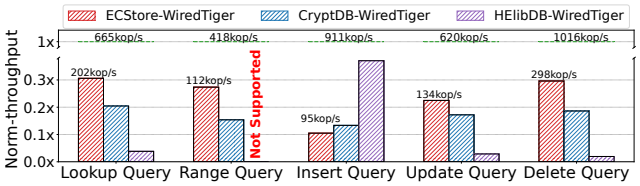


Fig. 11: Comparison of EDB throughput. The green dashed line is vanilla (unencrypted) WiredTiger's averaged throughput.

by the CDF of range query latency in Figure 9 (a). In the TPC-C workload, these range queries involve uniformly accessing 20% to 80% of random elements; longer query ranges led to higher latency (for scanning the leaf nodes and authentication). ECSTORE's latency distribution is more closely aligned with the vanilla baseline compared to CryptDB. HElibDB was not evaluated as it does not support range queries.

Sensitivity Study. Figure 9 (b) shows the maximum server throughput, achieved by saturating the server with up to 10 clients while varying database sizes on the YCSB. Initially, when the database size was small, both ECSTORE and the baselines could fit the data in memory. UC-ECSTORE had 1.2x higher throughput than ECSTORE because ECSTORE requires additional $O(logM)$ PGM lookups for compression, where M is the number of linear models used in the PGM. As the database size increased, CryptDB and UC-ECSTORE could not fit in memory anymore. Once they began accessing persistent storage for reads, their throughput dropped significantly. ECSTORE maintained high throughput until the compressed index surpassed the memory capacity. In this phase, ECSTORE achieved 2.9x to 10.5x higher throughput over the baselines. For larger database sizes, the throughput remained low and stable as the overhead was dominated by costly accesses to persistent storage. This experiment shows the effectiveness of EDB index compression.

For the mixed YCSB workloads (A-F), as depicted in Figure 12, ECSTORE achieved 202k operations per second on the YCSB-C (read-only) workload, consistent with the results observed in Figure 11. For other workloads excluding YCSB-F, ECSTORE consistently achieved higher throughput than CryptDB. This is because CryptDB's design necessitates frequent switching between multiple cryptographic indexes in handling mixed workloads, whereas ECSTORE tackles all index operations via a single ECTREE index without needing index switches. For YCSB-F, HElibDB had a higher peak throughput than other systems owing to its append-only design optimized for inserts. Besides, we report the number of adjustments and the ratio of time spent on adjustments to the running time. Table IV shows the rarity of adjustments of ECTREE on TPC-C, YCSB, and our customized KV workload

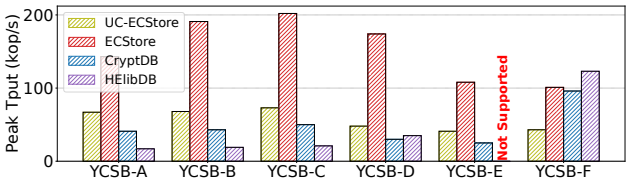


Fig. 12: Comparison of throughput on various EDBs using YCSB. Note that HElibDB does not support range queries.

Datasets	TPC-C	YCSB-D	YCSB-F	Custom. KV
# / %	27 / 13%	11 / 23%	14 / 26%	12 / 33%

TABLE IV: The number and time ratio of ECTREE adjustments.

(which is non-uniform), and the time used in adjustments is moderate compared with the overall running time.

To choose the best adjustment parameters, we also give an analysis for parameters in ECSTORE, i.e., the expansion ratio threshold α and the hallucination ratio threshold β discussed in §IV-B. The results illustrated in Figure 14 demonstrate that ECSTORE consistently achieves better throughput using the default parameter values compared to alternative settings. This outcome validates the optimality of the default values selected by ECSTORE. Notably, our findings indicate that ECTREE's adjustment is not particularly sensitive to these parameters. For instance, Figure 14 (a) shows a maximum performance degradation of only 12% for α = 1.5 in contrast to the scenario when α = 2. A similar trend is observed for β, as depicted in Figure 14 (b). Therefore, these results indicate that ECSTORE's performance is stable.

Notably, Figure 10 (b) demonstrates that ECTREE compression yields stable benefits. Under both uniform and Zipfian data workloads, ECSTORE consistently achieved a high compression ratio. This stability is attributed to the use of the same cryptographic hash functions in constructing subfilters, which ensures that the non-zero bits are uniformly ditributed regardless of the plaintext data distribution. The variations in index size for the same dataset (e.g., 20M) under different distributions arise primarily from the random placement of non-zero bits in subfilters. This variability affects the number of PGM indexes required to compress the index nodes, resulting in slight fluctuations in the final index size. Nonetheless, ECSTORE consistently achieves a high compression ratio across diverse workloads.

In sum, ECSTORE stands out by supporting a full set of index operations with lower latency and higher throughput than existing EDBs. ECSTORE favors read performance and tolerates moderate write performance downgrades. ECSTORE is most suitable for applications that prioritize read performance and require end-to-end data confidentiality, such as financial statistics [61] and medical analysis [62].

B. Scale-out Performance

ECSTORE implements a coarse-grained scheme [73] to distribute an EDB across multiple servers (i.e., scale-out). For a NoSQL instance, ECSTORE first allocates elements (comprising OWF-transformed keys and homomorphically encrypted values) to servers based on a range-based partitioning function for the keys. Then each server constructs ECTREE individually

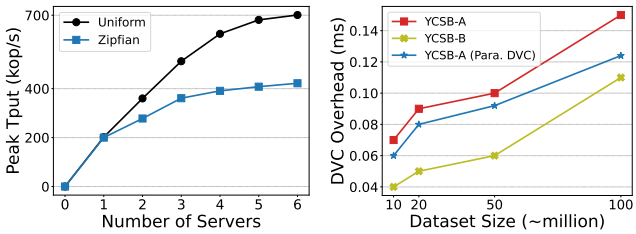


Fig. 13: (a) Scalability of ECTREE on YCSB-C by increasing the number of servers. (b) Overhead of DVC check on two high-update KV workloads.

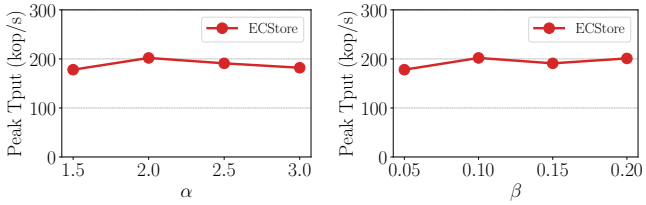


Fig. 14: Parameter sensitivity in index adjustment: (a) The effect of the expansion ratio α. (b) The effect of the hallucination ratio β.

for its assigned keys. The client maintains separate ECTREE caches and version tables for elements in each server, and uses the same partitioning function to decide which server should handle a specific query. Based on it, the client can perform a full set of operations listed in §II-C.

Figure 13 (a) shows the scalability of ECSTORE using up to 6 servers and 10 clients on a read-only (YCSB-C) workload. We scaled ECSTORE by partitioning a YCSB dataset (with 600M keys) with the above-mentioned approach. Under a uniform distribution, ECSTORE's peak throughput scaled to 1.8x and 2.5x by using 2 and 3 servers, respectively. When employing a skewed query distribution (Zipfian), ECSTORE just reached 422kop/s with 6 servers, since a substantial proportion of queries (nearly 40%) were sent to a single server, throttling the entire query system.

C. Robustness to Integrity Breaches

In preceding evaluations, we assume that the server adheres to our protocol in an honest but curious manner (with no active attacks). Next, we tested the performance under malicious attacks that manipulate query results. We utilized the metric of *effective* throughput, in the sense that we log only correct results and disregard manipulated results. Specifically, we simulated attacks by randomly altering half of the query results on the YCSB-C workload. These alterations included modifications, replays, and deletions. The results depicted in Figure 15(a) show that ECSTORE sustained a stable effective throughput with carefully designed MACs and ECTREE root verification (§IV-D). In contrast, the effective throughput of other baseline systems experienced a significant drop to 50% under malicious manipulations as they lack defenses against integrity breaches. This stark contrast highlights ECSTORE's goal of achieving both data confidentiality and query integrity simultaneously.

D. Lessons Learned

Supporting Variable-length Keys. Prior cryptographic index constructions [8], [12], [13] support only fixed-length keys

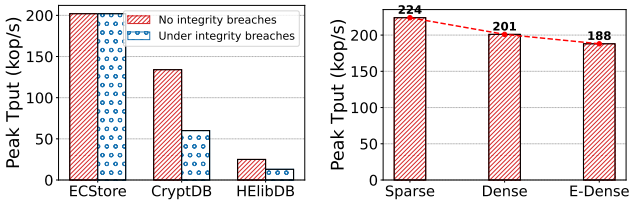


Fig. 15: (a) Robustness evaluation: the *effective* throughput of different systems under integrity breachs. (b) Performance of ECSTORE on workloads with varying data densities: Sparse (10% non-zero data), Dense (50% non-zero data, the **default**), and E-Dense (90% non-zero data).

due to constraints inherent in encryption schemes. To support variable-length keys, prior work typically stores a fat pointer in the leaf node of an index instead of the actual key, encoding its size and position [37]. In contrast, ECSTORE employs cryptographic hashing to consistently map variable-length keys into fixed-size subfilters, enabling an EDB index that natively supports variable-length keys. Cryptographic hash functions like SHA-256 are widely adopted in security-critical applications such as password managers [74] and phishing detectors [75]. As discussed in §III-B, ensuring the onewayness of primary keys is sufficiently secure in ECSTORE, given that (1) keys are exclusively used for lookups without decryption and (2) keys remain unique, with the assumption that the server does not collude with honest clients to perform write queries that could enable pre-image [52] or dictionary attacks [53] to reverse cryptographic hashes.

Extending Support for Multi-Dimensional Searches. ECTREE is designed for single-attribute indexing. To enable encrypted searches across multiple attributes (dimensions), as in graph databases [68], ECTREE should be extended to integrate with multi-dimensional indexes. For efficient exact matching across multiple columns, integrating ECTREE with a multi-dimensional binary search tree (MD-BST) [76] can be promising. MD-BST employs a level-based dimension-splitting strategy: sorting elements by one dimension, selecting the median, alternating between columns at each level and repeating the process. Since ECTREE already maintains order information for subfilters at its bottom level (via the client-side cache, §IV-B), it can directly adopt the MD-BST's dimension-splitting approach. For more complex queries, such as nearest neighbor searches, ECTREE can be combined with k-dimensional trees (*k-d* trees) [77]. Overall, given that ECTREE already organizes key order at the bottom level and multi-dimensional indexes require order of keys, extending ECTREE to support multi-dimensional searches is feasible and adaptable to specific query needs across multiple columns.

TEE-based and MPC-based Approaches. Trusted Execution Environments (TEEs), particularly Intel SGX [78], provide a secure memory space called enclave that provides data confidentiality and code integrity guarantees while supporting arbitrary computation at processor speeds. Opaque [79] is a representative TEE-based system that isolates sensitive SQL operations (e.g., search predicates) within SGX enclaves and hiding search access patterns by developing oblivious operators. While being efficient, Opaque relies on the trust on hardware vendors, while ECSTORE does not require such hardware trust. Multi-Party Computation (MPC)-based approaches, like Shortcut [80] and HybrTC [38], provide strong cryptographic guarantees but often rely on a semi-honest assumption, where parties are expected to follow the protocols honestly. Besides, MPC protocols typically incur high computational costs and need frequent communication, which can scale logarithmically or even linearly with the database size [80]. In contrast, ECSTORE provides query integrity guarantees under a strict threat model where query results may be manipulated (§III-B) and ECSTORE requires only a single interaction round.

Metadata-hiding Encrypted Search. Existing work on encrypted search can be broadly categorized into two groups. The first category, including ECSTORE, focuses on building an EDB that is practical in terms of functionality and efficiency but may leak metadata, such as search access patterns (e.g., the frequency of queried elements). The second category [19], [50], [81], [82] is expensive but not leaks metadata. A typical solution in this category is Oblivious RAM (ORAM) [58], [83], [84], a cryptographic tool that allows a client to access data from a server without revealing access patterns. However, despite the polylogarithmic complexity of ORAM, the cost of even the most practical ORAM schemes remains prohibitively high due to the need to process each operation on a bucket of data, which can contain millions of data elements. Other solutions, such as distributed trust [8] and private information retrieval [51], either rely on stringent security assumptions or compromise query functionality.

In summary, ECSTORE is orthogonal to recent advances in cryptography that aim for 'optimal' security by concealing all forms of metadata. While ECSTORE does not offer perfect security—since it leaks limited side-channel information related to specific query operations, such as equality checks and sorting—it achieves a practical balance between security and efficiency. We envision future integration of ECSTORE with advanced side-channel mitigation techniques to further strengthen its security guarantees, potentially at the cost of modest performance overhead.

VII. CONCLUSION

We present ECSTORE, the first encrypted database to support a full set of index operations with high efficiency. At its core is ECTREE, a logarithmic-complexity, compressible EDB index for encrypted data search. ECTREE's unique Merkle-tree-like structure allows query authentication, enabling ECSTORE to provide both data confidentiality and query integrity within a unified system. Extensive results on SQL and NoSQL instances demonstrate that ECSTORE outperforms existing baselines in terms of efficiency, scalability, and robustness. ECSTORE is well-suited for deployment in various security-critical applications, such as anonymous messaging and online streaming applications, aiding compliance with data protection regulations and enhancing data privacy.

ACKNOWLEDGMENTS

The work is supported in part by National Key R&D Program of China (No.2023YFB4503905), the 10th Young Elite Scientists Sponsorship Program by CAST (2024QNRC001),

HK RGC RIF (R7030-22), a Huawei Flagship Research Grant in 2023, HK RGC GRF (Ref: 17208223 & 17204424), and the HKU-CAS Joint Laboratory for Intelligent System Software.

REFERENCES

[1] I. Dropbox, "Dropbox," *http://www. dropbox. com*, 2014.

[2] K. Wilson and K. Wilson, "Onedrive," *Everyday Computing with Windows 8.1*, pp. 71–74, 2015.

[3] M. Mulazzani, S. Schrittwieser, M. Leithner, M. Huber, and E. Weippl, "Dark clouds on the horizon: Using cloud storage as attack vector and online slack space," in *20th USENIX Security Symposium (USENIX Security 11)*, 2011.

[4] A. Agarwal, M. Khari, and R. Singh, "Detection of ddos attack using deep learning model in cloud storage application," *Wireless Personal Communications*, pp. 1–21, 2021.

[5] K. Xue, W. Chen, W. Li, J. Hong, and P. Hong, "Combining data owner-side and cloud-side access control for encrypted cloud storage," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 8, pp. 2062–2074, 2018.

[6] (2023, May 15) Tiktok data breach timeline. [Online]. Available: https://firewalltimes.com/tiktok-data-breach-timeline/

[7] I. Governance. (2023) List of data breaches and cyber attacks in 2023. Accessed on November 2, 2023. [Online]. Available: https://www.itgovernance.co.uk/blog/list-of-data-breaches-and-cyber-attacks-in-2023

[8] E. Dauterman, E. Feng, E. Luo, R. A. Popa, and I. Stoica, "Dory: An encrypted search system with distributed trust," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 1101–1119.

[9] M. Bellare, M. Fischlin, A. O'Neill, and T. Ristenpart, "Deterministic encryption: Definitional equivalences and constructions without random oracles," in *Advances in Cryptology–CRYPTO 2008: 28th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2008. Proceedings 28*. Springer, 2008, pp. 360–378.

[10] J. Zhu, K. Cheng, J. Liu, and L. Guo, "Full encryption: An end-to-end encryption mechanism in gaussdb," in *Proceedings of the VLDB Endowment*, vol. 14, no. 12. VLDB Endowment, 2021, pp. 2811–2814. [Online]. Available: https://vldb.org/pvldb/vol14/p2811-guo.pdf

[11] Alibaba Cloud, "Data encryption of apsaradb rds," https://www.alibabacloud.com/help/en/rds/support/data-encryption, 2025, accessed: 2025-04-22.

[12] R. A. Popa, C. M. Redfield, N. Zeldovich, and H. Balakrishnan, "Cryptdb: Protecting confidentiality with encrypted query processing," in *Proceedings of the twenty-third ACM symposium on operating systems principles*, 2011, pp. 85–100.

[13] H. E. L. Contributors, "Helib: Homomorphic encryption library," https://github.com/homenc/HElib, 2021.

[14] R. Agrawal, J. Kiernan, R. Srikant, and Y. Xu, "Order preserving encryption for numeric data," in *Proceedings of the 2004 ACM SIGMOD international conference on Management of data*, 2004, pp. 563–574.

[15] W. Zheng, F. Li, R. A. Popa, I. Stoica, and R. Agarwal, "Minicrypt: Reconciling encryption and compression for big data stores," in *Proceedings of the Twelfth European Conference on Computer Systems*, 2017, pp. 191–204.

[16] M. Johnson, P. Ishwar, V. Prabhakaran, D. Schonberg, and K. Ramchandran, "On compressing encrypted data," *IEEE Transactions on Signal Processing*, vol. 52, no. 10, pp. 2992–3006, 2004.

[17] "Keybase," 2014, website: https://keybase.io/.

[18] "Tresorit," 2011, website: https://tresorit.com/.

[19] M. B. Aliyu, "Efficiency of boolean search strings for information retrieval," *American Journal of Engineering Research*, vol. 6, no. 11, pp. 216–222, 2017.

[20] B. R. Iyer and D. Wilhite, "Data compression support in databases," in *VLDB*, vol. 94, 1994, pp. 695–704.

[21] T. Kraska, A. Beutel, E. H. Chi, J. Dean, N. Polyzotis, and L. Zhu, "Consistency guarantees for parallel incremental data processing," in *Proceedings of the 2019 International Conference on Management of Data*. ACM, 2019, pp. 1271–1288. [Online]. Available: https://dl.acm.org/doi/10.1145/3299869.3319882

[22] C. Li, W. Ma, and L. Qin, "Efficient and accurate approximate query processing on data warehouses with learned indexes," in *Proceedings of the 2019 International Conference on Management of Data*. ACM, 2019, pp. 1717–1734. [Online]. Available: https://dl.acm.org/doi/10.1145/3299869.3319886

[23] Y. Xia, X. Yu, M. Butrovich, A. Pavlo, and S. Devadas, "Litmus: Towards a practical database management system with verifiable acid properties and transaction correctness," in *Proceedings of the 2022 International Conference on Management of Data, Philadelphia, PA, USA*, 2022, pp. 12–17.

[24] W. Zhou, Y. Cai, Y. Peng, S. Wang, K. Ma, and F. Li, "Veridb: An sgx-based verifiable database," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2182–2194.

[25] A. MySQL, "Mysql," 2001.

[26] R. Chopade and V. Pachghare, "Mongodb indexing for performance improvement," in *ICT Systems and Sustainability*. Springer, 2020, pp. 529–539.

[27] S. T. Leutenegger and D. Dias, "A modeling study of the tpc-c benchmark," *ACM Sigmod Record*, vol. 22, no. 2, pp. 22–31, 1993.

[28] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM symposium on Cloud computing*, 2010, pp. 143–154.

[29] T. Shen, J. Jiang, Y. Jiang, X. Chen, J. Qi, S. Zhao, F. Zhang, X. Luo, and H. Cui, "Daenet: making strong anonymity scale in a fully decentralized network," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2286–2303, 2021.

[30] H. Xu, Z. Cai, D. Takabi, and W. Li, "Audio-visual autoencoding for privacy-preserving video streaming," *IEEE Internet of Things Journal*, vol. 9, no. 3, pp. 1749–1761, 2021.

[31] D. Stehlé and R. Steinfeld, "Faster fully homomorphic encryption," in *Advances in Cryptology-ASIACRYPT 2010: 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings 16*. Springer, 2010, pp. 377–394.

[32] R. A. Popa, F. H. Li, and N. Zeldovich, "An ideal-security protocol for order-preserving encoding," in *2013 IEEE Symposium on Security and Privacy*. IEEE, 2013, pp. 463–477.

[33] D. Li, S. Lv, Y. Huang, Y. Liu, T. Li, Z. Liu, and L. Guo, "Frequency-hiding order-preserving encryption with small client storage," *Proceedings of the VLDB Endowment*, vol. 14, no. 13, pp. 3295–3307, 2021.

[34] D. X. Song, D. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *Proceeding 2000 IEEE symposium on security and privacy. S&P 2000*. IEEE, 2000, pp. 44–55.

[35] H. Corrigan-Gibbs and D. Kogan, "Private information retrieval with sublinear online time," in *Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I 39*. Springer, 2020, pp. 44–75.

[36] E. W. Weisstein, "Fermat's little theorem," *https://mathworld. wolfram. com/*, 2004.

[37] S. Golomb, "Run-length encodings (corresp.)," *IEEE transactions on information theory*, vol. 12, no. 3, pp. 399–401, 1966.

[38] P. Wu, J. Ning, J. Shen, H. Wang, and E.-C. Chang, "Hybrid trust multi-party computation with trusted execution environment," in *The Network and Distributed System Security (NDSS) Symposium*, 2022.

[39] E.-J. Goh, "Secure indexes," *Cryptology ePrint Archive*, 2003.

[40] V. Pappas, F. Krell, B. Vo, V. Kolesnikov, T. Malkin, S. G. Choi, W. George, A. Keromytis, and S. Bellovin, "Blind seer: A scalable private dbms," in *2014 IEEE Symposium on Security and Privacy*. IEEE, 2014, pp. 359–374.

[41] M. A. Bassiouni, "Data compression in scientific and statistical databases," *IEEE Transactions on Software Engineering*, no. 10, pp. 1047–1058, 1985.

[42] W. Cai, Z. Cai, X. Zhang, X. Wang, and M. Li, "A novel learnable dictionary encoding layer for end-to-end language identification," in *2018 IEEE international conference on acoustics, speech and signal processing (ICASSP)*. IEEE, 2018, pp. 5189–5193.

[43] F. Angius, M. Gerla, and G. Pau, "Bloogo: Bloom filter based gossip algorithm for wireless ndn," in *Proceedings of the 1st ACM workshop on Emerging Name-Oriented Mobile Networking Design-Architecture, Algorithms, and Applications*, 2012, pp. 25–30.

[44] A. W. Appel and D. McAllester, "An indexed model of recursive types for foundational proof-carrying code," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 23, no. 5, pp. 657–683, 2001.

[45] P. Ferragina and G. Vinciguerra, "The pgm-index: a fully-dynamic compressed learned index with provable worst-case bounds," *Proceedings of the VLDB Endowment*, vol. 13, no. 8, pp. 1162–1175, 2020.

[46] S. Gueron, S. Johnson, and J. Walker, "Sha-512/256," in *2011 Eighth International Conference on Information Technology: New Generations*. IEEE, 2011, pp. 354–358.

[47] X. Yi, R. Paulet, E. Bertino, X. Yi, R. Paulet, and E. Bertino, *Homomorphic encryption*. Springer, 2014.
[48] A. Lakshman and P. Malik, "Cassandra: a decentralized structured storage system," *ACM SIGOPS operating systems review*, vol. 44, no. 2, pp. 35–40, 2010.
[49] W. Chen, T. Hoang, J. Guajardo, and A. A. Yavuz, "Titanium: A metadata-hiding file-sharing system with malicious security," *Cryptology ePrint Archive*, 2022.
[50] E. Dauterman, E. Feng, E. Luo, R. A. Popa, and I. Stoica, "Dory: An encrypted search system with distributed trust," in *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, 2020, pp. 1101–1119.
[51] Y. Ma, K. Zhong, T. Rabin, and S. Angel, "Incremental {Offline/Online}{PIR}," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 1741–1758.
[52] M. Amy, O. Di Matteo, V. Gheorghiu, M. Mosca, A. Parent, and J. Schanck, "Estimating the cost of generic quantum pre-image attacks on sha-2 and sha-3," in *International Conference on Selected Areas in Cryptography*. Springer, 2016, pp. 317–337.
[53] R. R. Al-Dahhan, Q. Shi, G. M. Lee, and K. Kifayat, "Survey on revocation in ciphertext-policy attribute-based encryption," *Sensors*, vol. 19, no. 7, p. 1695, 2019.
[54] R. Poddar, S. Kalra, A. Yanai, R. Deng, R. A. Popa, and J. M. Hellerstein, "Senate: a {Maliciously-Secure}{MPC} platform for collaborative analytics," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2129–2146.
[55] P. Wu, J. Ning, J. Shen, H. Wang, and E.-C. Chang, "Hybrid trust multi-party computation with trusted execution environment."
[56] C. Fontaine and F. Galand, "A survey of homomorphic encryption for nonspecialists," *EURASIP Journal on Information Security*, vol. 2007, pp. 1–10, 2007.
[57] S. Chen and Q. Jin, "Persistent b+-trees in non-volatile main memory," *Proceedings of the VLDB Endowment*, vol. 8, no. 7, pp. 786–797, 2015.
[58] C. W. Fletcher, L. Ren, A. Kwon, M. Van Dijk, and S. Devadas, "Freecursive oram: [nearly] free recursion and integrity verification for position-based oblivious ram," in *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2015, pp. 103–116.
[59] F. Zhang, W. Wan, C. Zhang, J. Zhai, Y. Chai, H. Li, and X. Du, "Compressdb: Enabling efficient compressed data direct processing for various databases," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 1655–1669.
[60] F. Chen, M. P. Mesnier, and S. Hahn, "Client-aware cloud storage," in *2014 30th Symposium on Mass Storage Systems and Technologies (MSST)*. IEEE, 2014, pp. 1–12.
[61] D. Ruppert and D. S. Matteson, *Statistics and data analysis for financial engineering*. Springer, 2011, vol. 13.
[62] A.-B. Haidich, "Meta-analysis in medical research," *Hippokratia*, vol. 14, no. Suppl 1, p. 29, 2010.
[63] D. Sarkar, A. De, C. K. Chanda, and S. Goswami, "Kruskal's maximal spanning tree algorithm for optimizing distribution network topology to improve voltage stability," *Electric Power Components and Systems*, vol. 43, no. 17, pp. 1921–1930, 2015.
[64] M. Szydlo, "Merkle tree traversal in log space and time," in *Advances in Cryptology-EUROCRYPT 2004: International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004. Proceedings 23*. Springer, 2004, pp. 541–554.
[65] X. Wang, H. Zhao, and J. Zhu, "Grpc: A communication cooperation mechanism in distributed systems," *ACM SIGOPS Operating Systems Review*, vol. 27, no. 3, pp. 75–86, 1993.
[66] E. Käsper and P. Schwabe, "Faster and timing-attack resistant aes-gcm," in *International Workshop on Cryptographic Hardware and Embedded Systems*. Springer, 2009, pp. 1–17.
[67] S. Burckhardt *et al.*, "Principles of eventual consistency," *Foundations and Trends® in Programming Languages*, vol. 1, no. 1-2, pp. 1–150, 2014.
[68] R. Angles and C. Gutierrez, "Survey of graph database models," *ACM Computing Surveys (CSUR)*, vol. 40, no. 1, pp. 1–39, 2008.
[69] W. Almesberger *et al.*, "Linux network traffic control—implementation overview," 1999.
[70] E. W. Weisstein, "Fermat's little theorem," *https://mathworld. wolfram. com/*, 2004.
[71] X. Wei, R. Chen, and H. Chen, "Fast {RDMA-based} ordered {Key-Value} store using remote learned cache," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 117–135.
[72] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, and M. Paleczny, "Workload analysis of a large-scale key-value store," in *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*, 2012, pp. 53–64.
[73] T. Ziegler, S. Tumkur Vani, C. Binnig, R. Fonseca, and T. Kraska, "Designing distributed tree-based index structures for fast rdma-capable networks," in *Proceedings of the 2019 International Conference on Management of Data*, 2019, pp. 741–758.
[74] S. Assiri and B. Cambou, "Homomorphic password manager using multiple-hash with puf," in *Advances in Information and Communication: Proceedings of the 2021 Future of Information and Communication Conference (FICC), Volume 1*. Springer, 2021, pp. 772–792.
[75] M. Khonji, Y. Iraqi, and A. Jones, "Phishing detection: a literature survey," *IEEE Communications Surveys & Tutorials*, vol. 15, no. 4, pp. 2091–2121, 2013.
[76] D. Dereniowski, P. Gordinowicz, and K. Wróbel, "On multidimensional generalization of binary search," *arXiv preprint arXiv:2404.13193*, 2024.
[77] K. Zhou, Q. Hou, R. Wang, and B. Guo, "Real-time kd-tree construction on graphics hardware," *ACM Transactions on Graphics (TOG)*, vol. 27, no. 5, pp. 1–11, 2008.
[78] V. Costan, "Intel sgx explained," *IACR Cryptol, EPrint Arch*, 2016.
[79] W. Zheng, A. Dave, J. G. Beekman, R. A. Popa, J. E. Gonzalez, and I. Stoica, "Opaque: An oblivious and encrypted distributed analytics platform," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 283–298.
[80] P. Zhou, X. Guo, P. Chen, T. Li, S. Lv, and Z. Liu, "Shortcut: Making mpc-based collaborative analytics efficient on dynamic databases," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 854–868.
[81] A. H. Lashkari, F. Mahdavi, and V. Ghomi, "A boolean model in information retrieval for search engines," in *2009 International Conference on Information Management and Engineering*. IEEE, 2009, pp. 385–389.
[82] M. He, J. Zhang, G. Zeng, and S. M. Yiu, "A privacy-preserving multi-pattern matching scheme for searching strings in cloud database," in *2017 15th Annual Conference on Privacy, Security and Trust (PST)*. IEEE, 2017, pp. 293–29 309.
[83] J. Doerner and A. Shelat, "Scaling oram for secure computation," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 523–535.
[84] E. Stefanov, M. v. Dijk, E. Shi, T.-H. H. Chan, C. Fletcher, L. Ren, X. Yu, and S. Devadas, "Path oram: an extremely simple oblivious ram protocol," *Journal of the ACM (JACM)*, vol. 65, no. 4, pp. 1–26, 2018.



Tianxiang Shen is a principal engineer in the Huawei OS kernel lab. His research interests are in distributed system and trusted computing. He leveraged trusted computing tools to implement secure and robust distributed computing systems. He has been founded by CAST's Young Elite Scientists Sponsorship Program.



Ji Qi is an assistant professor at Institute of Software, Chinese Academy of Sciences. He received Ph.D. degree in Department of Computer Science at The University of Hong Kong, and the Master degree in School of Software at Tsinghua University. Before that he received the Bachelor degree in School of Software at Beijing Institute of Technology.



Ning Jia is the director of Huawei OS kernel Lab. His research interests include operating system, software-hardware co-design and system security. He is leading a team to develop a general-purpose microkernel. He is a founding member of the technical steering committee of OpenHarmony opensource community.



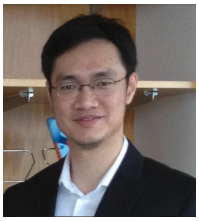
Haoze Song is a Ph.D. student in the department of computer science at the University of Hong Kong. He received his bachelor's degree in Computer Science from the University of Science and Technology of China with honors. His current research interests are in the areas of data management in cloud computing environments, distributed transactions, and consensus protocol.



Xiapu Luo received the Ph.D. degree from the Hong Kong Polytechnic University. He is an associate professor with the Department of Computing, the Hong Kong Polytechnic University. He spent two years at the Georgia Institute of Technology as a postdoctoral research fellow. His current research interests include network and system security, blockchain and smart contract, mobile, and IoT security.



Sen Wang received the B.S. degree from USTC in 2005, the M.S. degree from the Chinese Academy of Sciences (CAS) in 2008, and the Ph.D. degree from Tsinghua University in 2014. From 2014 to 2019, he was a lecturer and then an associate professor at Chongqing University. Currently, he is a senior researcher at Huawei, Hongkong. His research interests include information-centric networking, Federated Learning and AI for System.



Heming Cui (Member, IEEE) is an associate professor in computer science of HKU. His research interests include operating systems, programming languages, distributed systems, and cloud computing, with a particular focus on building software infrastructures and tools to improve reliability and security of real-world software. Homepage: https://i.cs.hku.hk/heming/.